

The Illusion of Competence

AI assistance, coding skills, and the hidden cost of easy answers

artifact path

ask AI → get code → task passes

VS.

learning path

predict → try → fail → debug → explain → retain

Which path did you use the last time AI helped you code?

Shen & Tamkin, Anthropic Research, 2026

The visible win hides the invisible cost

WHAT ORGANIZATIONS SEE

faster drafts
completed tickets
fewer syntax errors



WHAT MAY BE MISSING

weaker mental model
less debugging practice
less rejection of plausible
wrong code

“How many of us have approved or shipped code this week that we could not fully explain line by line?”

Oversight is not a job title. It is a skill.

If AI writes more code, human review becomes more important, not less.

AI generates



human reviews



system ships

REVIEW REQUIRES

- detect structural errors
- read unfamiliar code
- understand library semantics
- say no to plausible output

The paper asks whether AI-assisted work builds the review skill or bypasses the practice loop.

The experiment as a pipeline

The design separates “finished the task” from “retained the skill.”

1 Recruit

52 Python developers;
Trio unfamiliar

2 Calibrate

warm-up task without AI

3 Randomize

two Trio tasks; AI sidebar
or no AI

4 Evaluate

final quiz with **no AI**

AI CONDITION

chat-based coding assistant in the sidebar; it could see current code and produce correct code

NO-AI CONDITION

same learning materials and coding tasks, but no assistant during the Trio stage

What exactly did they ask developers to learn?

Trio was a deliberately unfamiliar async library, not a Python syntax test.

Task 1

Write a timer that prints each passing second while other functions run.

TARGET CONCEPTS

nurseries · starting tasks · concurrent functions

Task 2

Implement record retrieval that handles missing-record errors.

TARGET CONCEPTS

error handling · memory channels · result flow

The task simulates workplace onboarding: a short tutorial, starter code, then a small feature using a new library.

How did they measure learning?

The quiz measured whether participants could supervise code without the assistant.

14

questions

27 total points

DEBUGGING

identify why code fails

7

Trio concepts

covered by the two tasks

CODE READING

understand unfamiliar code

0

AI access

during the quiz

CONCEPTUAL

know the library model

They excluded code-writing questions to reduce Python syntax confounds; the quiz rubric was submitted in preregistration.

How did they conclude there was a learning penalty?

Random assignment makes the comparison: same task, different access to AI.

GROUPS

- 26 AI participants
- 26 no-AI participants

PRODUCTIVITY OUTCOME

completion time difference: about two minutes
not significant (p=0.391)

LEARNING OUTCOME

quiz gap: 4.15 points on a 27-point quiz
significant (d=0.738, p=0.010)

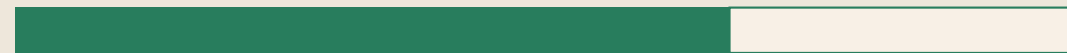
Their claim is not “AI users failed the task.” It is: after the same learning task, AI access predicted lower unassisted mastery.

The productivity story breaks

Lower scores, no reliable speed win.

THE RETENTION CRASH

Hand-coding group score:



67%

AI-assisted group score:



50%

Statistically significant 17 pp learning drop

Average time gain: about two minutes, not statistically significant.

A finished task did not translate into retained mastery.

Debugging is where the gap bites

debugging questions

were the largest group gap.

No-AI learners hit errors



they repaired those errors



repair trained judgment

OBSERVED PATTERN

Largest gap: debugging.

Smallest gap: code reading.

CAVEAT

Plausible mechanism, not proven mediation.

Where did the time savings go?

Generation is fast. The workflow is not.

11

MINUTES

some participants spent composing queries

15

QUESTIONS

some participants asked the assistant



CHECKING

the workflow still included validation and fixing

The model may be working; the learner may not be practicing.

Not all AI use is the same

The difference is whether AI removes practice or scaffolds it.

OFFLOADING PATTERNS

- delegation
- progressive reliance
- AI debugging

asks the model to write, diagnose, or finish the hard part

LEARNING PATTERNS

- conceptual inquiry
- code + explanation
- self-checking

keeps prediction, explanation, or judgment in the human loop

The interaction clusters are descriptive; they do not by themselves prove causal effects.

Same assistant. Different practice loop.

artifact path

DELEGATION

1. User: write the solution
2. AI: here is code
3. User: paste / pass

Outcome: cannot debug variant

VS.

learning path

LEARNING

1. User: asks concept
2. User: predicts behavior
3. User: tries, hits error

Outcome: can debug variant

Which persona was your last AI session?

Think of the last time AI helped you code.

1

Did I predict before asking?

2

Did I debug anything myself?

3

If the model disappeared, what could I still explain?

Pair-share: delegation, debugging-by-AI, or learning-oriented?

Generator or coach?

AI tools can preserve effort instead of bypassing it.

Ask-before-answer

elicit a hypothesis first

Hint-before-fix

support debugging without taking it over

Explain-after-generation

require comprehension

Mode clarity

ship mode versus learn mode

Which lever would annoy you in production but help you while learning?

When delegation felt productive but did not teach enough

My Tonic profiling struggle looked like the paper's warning sign.

ASK

AI help produced runs, tables,
and dashboards.



INSPECT

Comparisons were confounded;
hot-path timers changed the
system.



REFRAME

Matched regimes;
instrumentation-off throughput;
perf evidence.

WHAT FINALLY HELPED

bounded matrix
bucketed timers as diagnostic mode
buffer-policy controls
frame-pointer / perf reruns

**The learning happened when I challenged the artifact,
not when the artifact appeared.**

The coroutine bug worked because I steered the debugger

I asked Claude to debug, but I supplied the hypothesis and boundary.

MY DIRECTION

Focus on DSA alignment inside
coroutine-frame storage:
64-byte descriptor, 32-byte
completion.

CLAUDE'S WORK

inspect layout
challenge `alignas()`
trace where addresses are formed

FIX

over-allocate
compute aligned pointers once
cache hot-path addresses

DEBUGGING MODEL

$(\text{base} + \text{align} - 1) \& \sim(\text{align} - 1)$ preserved the
hardware contract when coroutine storage made naive
alignment suspect.

Learning-oriented delegation: outsource the search, keep
hypothesis, evidence, and judgment in the human loop.

One question to end on

If AI can produce the artifact, what should undergraduate CS education protect?

DISCUSSION

Which practice loop should remain mostly human: prediction, debugging, explanation, or design judgment?

Pick one, then propose a course policy that preserves it without pretending AI does not exist.

What should we still make students struggle through—and why?

AI-enhanced productivity is not a shortcut to competence

If future oversight depends on human expertise, our tools and habits must preserve the practice loop.

PRIMARY

Shen, Judy Hanwen, and Alex Tamkin. “How AI Impacts Skill Formation.” arXiv:2601.20245, 2026.

ARTICLE

Anthropic Research. “How AI assistance impacts the formation of coding skills.” 2026.

RELATED

Storey, Margaret-Anne. “How Generative and Agentic AI Shift Concern from Technical Debt to Cognitive Debt.” 2026.
margaretstorey.com/blog/2026/02/09/cognitive-debt/

RELATED

Sarkar, Advait. “Intention Is All You Need.” arXiv:2410.18851, 2024. arxiv.org/abs/2410.18851

VIDEO

Anthropic Research on AI and skill formation. YouTube reference: bp8dwCJF6iw.

CONTEXT

Prior AI-productivity, critical-thinking, and cognitive-offloading work cited in the Anthropic article.